Q1 (a) Raft State Machine Replication & Log Matching:

Log Matching Property: If two logs contain an entry with the identical index and term, then:

1. They store the same state machine command.
2. Their logs are completely identical in all preceding entries up to that index.

Commit Safety Rule for Older Terms:

A Raft leader cannot commit an entry from an older term simply by counting replicas.
Instead, the leader must commit an entry from its CURRENT term by replicating it to a majority. Once an entry from the current term is committed, the Log Matching Property guarantees that all prior entries from previous terms are committed indirectly. This prevents subtle race conditions where an old leader rejoins after network partition and overwrites already replicated entries.

Q1 (b) Multi-Paxos vs Raft Comparison:

Steady-State Message Rounds: Both Multi-Paxos and Raft require exactly 1 RTT once a stable leader is active (Raft Leader sends AppendEntries RPCs; Multi-Paxos executes Phase 2 Accept/Accepted).

Leader Failure Recovery Complexity: In Raft, leader election ensures that the candidate with the most up-to-date log (higher last term, longer log) wins the majority vote. Thus, the new leader already contains all committed entries and needs zero catch-up before serving client write requests.
In Multi-Paxos, any node can become leader, so it must run Phase 1 (Prepare/Promise) across all slots to discover uncommitted proposals from previous leaders, leading to significantly higher recovery complexity.

Q2 (a) Strict 2-Phase Locking vs Rigorous 2PL & WFG Deadlock:

Strict 2PL: All exclusive (write) locks held by transaction T must be retained until commit/abort.
Shared (read) locks can be released earlier. In Rigorous 2PL, ALL locks (shared + exclusive) are held to end.
Serializability: Strict 2PL guarantees serializability because transactions do not release write locks
before reaching their lock point, precluding cycles in conflict serialization graphs.
Avoiding Cascading Aborts: Since uncommitted write locks are held until commit, other transactions
cannot read dirty data from T. If T aborts, no other active transaction has read its intermediate writes.
Wait-For Graph (WFG): Directed graph G = (V, E) where vertices are transactions. Edge Ti -> Tj means
Ti is blocked waiting for a lock held by Tj. A cycle in WFG indicates a Deadlock (detected via DFS).

Q2 (b) Write Skew Anomaly under Snapshot Isolation (SI):

Scenario: Hospital constraint requires Active Doctors >= 1. Initial State: D1 = Active, D2 = Active.

Time t1: Transaction T1 reads snapshot (D1=Active, D2=Active). Active count = 2.

Time t2: Transaction T2 reads snapshot (D1=Active, D2=Active). Active count = 2.

Time t3: T1 sets D1 = Inactive (since count >= 1 is satisfied) and commits.

Time t4: T2 sets D2 = Inactive (since count >= 1 was satisfied on its snapshot) and commits.

Since T1 writes to D1 and T2 writes to D2, their write sets are disjoint! SI first-committer-wins rule
does not trigger an abort. Both commit, leaving Active Doctors = 0 (Violation of invariant!).
Serializable Snapshot Isolation (SSI) Solution: Tracks rw-antidependencies (T1 read D2 before T2 wrote it).
SSI aborts T2 upon detecting consecutive rw-antidependency edges forming a dangerous structure.

Q3 (a) Consistent Hashing with Virtual Nodes & Key Migration:

Keys and nodes are mapped to a 32-bit ring [0, 2^32 - 1]. Each physical node has V virtual nodes. Virtual nodes bound partition variance to O(1/V). When adding 1 node to an N-node cluster, the new node takes a fraction 1/(N + 1) of the hash ring from existing nodes. Therefore, exactly a fraction 1/(N + 1) of total keys migrate (i.e. K / (N + 1) keys), avoiding O(K) rehashing.

Q3 (b) Quorums, Vector Clocks & Last-Write-Wins (LWW):

Quorum Condition: Strong consistency requires R + W > N. By Pigeonhole Principle, read set R and write set W overlap in at least (R + W - N) nodes, guaranteeing at least one node has latest version. Vector clocks track causal order: VC(A) <= VC(B). Incomparable vectors indicate concurrent conflict. LWW Hazard: Relies on physical wall clocks. NTP clock skew/drift silently overwrites newer updates.

Q4 (a) Failure Detectors & FLP Impossibility:

Chandra-Toueg <>W Failure Detector: 1) Weak Completeness: every crashed node is eventually suspected. 2) Eventual Weak Accuracy: eventually some correct process is never suspected by any correct node. FLP Impossibility states consensus cannot terminate deterministically in asynchronous systems with even 1 crash failure. <>W circumvents FLP by adding partial synchrony during which consensus terminates.

Q4 (b) PBFT 3f + 1 Derivation:

In an asynchronous system with f Byzantine nodes: To guarantee liveness, the system must make progress after N - f responses. Among these N - f responses, up to f could be from Byzantine nodes lying. Number of guaranteed correct responses is (N - f) - f = N - 2f. To ensure correct nodes outvote Byzantine: N - 2f > f ===> N > 3f ===> N >= 3f + 1 nodes required. (Contrasts with crash-only N >= 2f + 1).